

1、signMask

需要返回只有最高位是 1，其余 31 位全为 0 的 32 位整数的掩码，所以用 1 左移 31 位，低位补 0。

```
* signMask - return a mask with only the most significant bit set
(0x80000000)
* Legal ops: ! ~ & ^ | + << >>
int signMask(void) {
    return 1 << 31;
}
```

2、bitXOR

由于只能用 &（位与）和 ~（按位取反）实现异或，可以从集合角度理解：异或表示两个位不相等，即 $(x|y)$ 且不同时为 1。先写成 $(x|y) \& \sim(x \& y)$ ，再利用德摩根定律将 $x|y$ 写成 $\sim(\sim x \& \sim y)$ ，于是得到下方代码。

```
* bitXor - x^y using only ~ and &
* Example: bitXor(4, 5) = 1, bitXor(7, 7) = 0
* Legal ops: ~ &
int bitXor(int x, int y) {
    return ~((x&y) & ~(\sim x&\sim y));
}
```

3、negativePart

要返回负数的相反数，正数则返回 0。由于负数符号位为 1，正数符号位为 0，考虑利用符号位构造掩码。具体思路见下方注释。

```
* negativePart - return -x if x < 0, otherwise return 0
* Examples: negativePart(-10) = 10, negativePart(5) = 0
* Legal ops: ! ~ & ^ | + << >>
int negativePart(int x){
    int m = x >> 31; // 掩码。负数则符号位 1，右移得 0xffffffff(-1)，
    即 111...1, 则 m&x 为 x 本身; 正数得到 00...0, m&x 为 0
    return (~x+1) & m; // x 取反+1 相当于 -x, 再&m 返回 -x
}
```

4、copyByteWithin

把 32 位整数 x 第 src 个字节复制到第 dst 个字节，其余三个字节保持不变。

16 进制中每个数字占 4bits，两个数字是一个 byte。

思路是尝试用 0 和 1 构造掩码，先把目标字节 dst 清成 0，其他字节不变。再把 byte src 提取出来，平移到 dst 的位置，最后两者相或，合并。

```
* copyByteWithin - copy byte src of x to byte dst, leaving all other
bytes unchanged
```

```

*   Bytes are numbered from 0 (least significant) to 3 (most
significant).
*   You can assume 0 <= src <= 3 and 0 <= dst <= 3.
*   Example: copyByteWithin(0x11223344, 0, 2) = 0x11443344
int copyByteWithin(int x, int src, int dst) {
    int origin=x;
    dst<<=3;src<<=3;//由于 16 进制中四个字节的起始位号是 16、12、8、0, byte k
的起始位=k*8, 相当于<<3
    int de_mask=~(0xff<<dst);//先构造只有 dst 字节是 FF、其余为 00 的掩码;
取反后变成 dst 字节为 00、其余为 FF,
    x&=de_mask;//相与, 把 dst 字节清零
    int s_mask=(0xff&(origin>>src))<<dst;//先把 src 字节右移到最右边, 而
0xff 只有最低一个字节是全 1, 相与后提取 src 字节, 然后左移到目标字节 dst
    x|=s_mask;//合并
    return x;
}

```

5、logicalShift

逻辑右移高位一律补 0（不同于算术右移时高位补原来的符号位）。思路是在 x 算数右移的基础上，构造掩码消除高位补充的符号位。

```

* logicalShift - shift x to the right by n bits, using a logical shift
*   Can assume that 0 <= n <= 31
*   Examples: logicalShift(0x87654321,4) = 0x08765432
int logicalShift(int x, int n) {
    int mask=~(((1<<31)>>n)<<1);//mask 让原本算数右移高位补的 1 清零。原来的
1+n 个 bit 的 1, 多了一位, 所以要再左移 1 位。
    return ((x>>n)&mask);
}

```

6、swapNibblePairs

要交换一个字节中高低四位。参考上面掩码的思路，构造对应掩码。先擦除高位，用位移把低位换到高位，反之同理，最后 OR 合并。

```

* swapNibblePairs - swap the low and high 4 bits within each byte of
x
*   Examples: swapNibblePairs(0xAB) = 0xBA
int swapNibblePairs(int x) {
    //参考上面掩码的思路, 构造 00011111
    //先擦除
    int m=0x0f;
    m=m|(m<<8);
    m=m|(m<<16);
    int highbits=(x&m)<<4;//把低位的换到高位。
    int lowbits=(x>>4)&m;//提取每个字节的高 4 位, 右移 4 位到低半字节位置, 并
去除负数时候右移高位产生的 1

```

```

x=highbits|lowbits;//拼接起来
return x;
}

```

7、secondLowestZeroBit

在 x 的 32 位二进制中，从低位往高位数，找到第二个值为 0 的位，返回一个只有该位为 1 的掩码。如果不足两个 0，就返回 0。

思路是先找到并消去最右边的一个 0，然后重复相同步骤找到第二个 0。最右边 0 的特征：右边都是 1，左边是任意数。利用进位的关系， $x+1$ 输出只有第一位 0 是 1，其右边都是 0，左边不变的掩码。再用 $\sim x \& (x+1)$ 来找到第一个最低位 0，因为只有最低位那个 0 的位置在 $\sim x$ 和 $x+1$ 里同时为 1

```

* secondLowestZeroBit - return a mask that marks the position of the
second least significant 0 bit
* Examples: secondLowestZeroBit(0xFFFFFFFF) = 0x4,
secondLowestZeroBit(0x7FFFFFFF) = 0
*           secondLowestZeroBit(-1) = 0
* Legal ops: ! ~ & ^ | + << >>
int secondLowestZeroBit(int x) {
    int low0 = ~x & (x + 1); // 找第一个最低位 0。如果 x 全为 1 没有 0，则
low0 为 0。
    int x2 = x | low0;       // 把第一个 0 变成 1，其余位不变
    return ~x2 & (x2 + 1);   // 重复操作，找新数的最低位 0，就是第二个
}

```

8、oddParity

偶数个 1，则输出 1。想到异或的特点：偶数个 1，返回 0，奇数个 1，返回 1。可以通过不断异或折半，最后得到一个 bit 表示 1 的奇偶性。

```

* oddParity - return the odd parity bit of x, that is,
*           when the number of 1s in the binary representation of x is even,
then the return 1, otherwise return 0.
* Examples: oddParity(5) = 1, oddParity(7) = 0
int oddParity(int x) {
    x^=x>>16;//高 16 位 ^ 低 16 位，结果存在低 16 位
    x^=x>>8;//不断异或折半
    x^=x>>4;
    x^=x>>2;
    x^=x>>1;//第 0 位代表 32 位的奇偶性
    x&=1;//1 的 32 位表示时，只有最后 1 位是 1，前 31 位都是 0，能作为掩码只保留
最后一位。想保留第 n 位就用 1 << n 做掩码，想清除某几位就用对应位为 0 的掩码
做按位与
    return !x;
}

```

9、rotateRightBits

把一个 32 位数循环右移 n 位，等价于把它从中间切开，右边移出的 n 位绕到左边。由于循环右移 32 位等于不变，先令 $m=n\&31$ 。再将 x 逻辑右移 m 位，把低 m 位移到高位，最后合并；这样也能正确处理 $n\geq 32$ 和 $n=0$ 。

```
* rotateRightBits - rotate x to right by n bits
*   you can assume n >= 0
*   Examples: rotateRightBits(0x12345678, 8) = 0x78123456//16 进制数，
每个数字占 4bits
int rotateRightBits(int x, int n) {
    int m = n & 31, mask = ~(((1 << 31) >> m) << 1); //m 等价于 n%32; mask
高 m 位为 0
    int right = (x >> m) & mask; //逻辑右移部分
    int leftShift = (32 + ~m) & 31; //等价于 (32-m)%32, m=0 时左移 0 位
    int left = x << leftShift; //低 m 位移到高位
    return left | right;
}
```

10、roundEvenPow2

要把非负整数 x 舍入到 2^n 的倍数。先数学建模。

令 $x=q \cdot M+r$, $M=2^n$, 商为 q 、余数为 r 。half= $2^{(n-1)}=M/2$, 是相邻两个倍数的中点。若 $r>half$ 则进位, $r<half$ 则舍去; $r=half$ 时按偶数商规则处理: q 为偶数不进位, q 为奇数则进位。该规则可写成 $r+(q\&1)\geq half+1$ 。

```
* roundEvenPow2 - round nonnegative x to the nearest multiple of 2^n.
*   If x is exactly halfway between two multiples, choose the multiple
whose quotient by 2^n is even. //偶数商不进位
*   You can assume 0 <= x <= 0x3fffffff and 1 <= n <= 16.
*   Examples: roundEvenPow2(10, 2) = 8, roundEvenPow2(14, 2) = 16
int roundEvenPow2(int x, int n) {
    int neg1=~0; //-1
    int n_minus1=n+neg1; //n-1
    int half = 1<<n_minus1; //2^(n-1)
    int half_plus1=half+1;
    int one_n=1<<n; //2^n
    int lowmask=one_n+neg1; //低 n 位全 1 掩码, 等价于 (1<<n)-1
    int r=x&lowmask; //余数, 即 x 的低 n 位, 用于判断是否需要给商加 1
    int q=x>>n; //取商
    int q_bit=q&1; //取商的最低位, 判断奇偶性。
    int a = r+q_bit;
    int diff=a+~half_plus1+1; //a-half-1, diff>=0 时需要进位
    int sign=(diff>>31)&1; //diff 的符号位
    int cin=!sign; //0 表示不进位, 1 表示进位
    return (q+cin)<<n; //商加上进位标志, 再左移, 还原为 2^n 的倍数
}
```

11、midpointTowardFirst

```

* midpointTowardFirst - return the exact mathematical midpoint (x+y)/2
* without overflow. If the exact midpoint lies halfway between two
* integers, choose the adjacent integer that is closer to the first
* argument x.
* Examples: midpointTowardFirst(4, 7) = 5,
*           midpointTowardFirst(7, 4) = 6

```

直接 `int sum = x + y` 可能超出 32 位有符号整数范围，必须无溢出。所以先分别对两个数除以 2，不会溢出，再根据最低位判断是否有进位或舍入，即右移 1 位相加 + 补低位进位。右移后数值范围是 -1073741824 到 1073741823，不会溢出。`((x & y) & 1)` 是看两数最低位相加是否需要进位。

```

int midpointTowardFirst(int x, int y) {
    int floor_avg = (x >> 1) + (y >> 1) + ((x & y) & 1);

```

`is_half` 为 1，则两数和为奇数，无法整除，中点是半整数有 0.5

```

    int is_half = (x ^ y) & 1;

```

还要判断半整数往那个方向舍入，即需要判断 `x` 和 `y` 的大小，用符号位比较。分为异号分支 | 同号分支判断。

```

    int sx = x >> 31;
    int sy = y >> 31;
    int diff = x + ~y + 1; // 计算 x - y
    int x_gt = (!sx & sy) | (!(sx ^ sy) & !(diff >> 31)); // 判断是否 x > y

```

半整数且 `x` 更大时，选大的那个

```

    return floor_avg + (is_half & x_gt);
}

```

12、isBetweenEitherOrder

```

* isBetweenEitherOrder - return 1 when x lies in the inclusive interval
whose
* endpoints are a and b. The endpoints may be given in either order.
* Example: isBetweenEitherOrder(5, 8, 3) = 1.

```

判断 `x` 是否在 `a` 和 `b` 的区间内，相当于分别判断 `x` 与 `a`，`x` 与 `b` 的大小关系。以 `x` 和 `a` 比大小为例，如果 `x > a`，返回 1，否则返回 0。由于直接进行减法比较可能溢出，所以根据 `x` 和 `a` 的符号位来分类，分为同号和异号。

`x` 和 `a` 异号：只有当 `x` 为非负数、`a` 为负数时，`x > a` 成立，这对应 `x` 的符号位为 0，`a` 的符号位为 1。

`x` 和 `a` 同号：此时不能直接判断 `x - a` 的符号位，因为如果 `x = a`，`x - a` 的符号位为 0，但 `x` 并不满足 `x > a`。

```

int isBetweenEitherOrder(int x, int a, int b) {
    int sign_x = x >> 31;
    int sign_a = a >> 31;
    int sign_b = b >> 31;

```

若 `x, a` 符号不同，只有 `x` 为非负数，`a` 为负数，满足 `x > a`。取两次非，使得结果只能是 0/1

```

    int xa_diff_sign = !((!sign_x) & sign_a);
    int xb_diff_sign = !((!sign_x) & sign_b);

```

同号时判断严格大于，如果直接判断 $x - a$ 的符号位，会把 $x = a$ 误判成 $x > a$ 。所以用 $x - a - 1 \geq 0$ ，符号位必然为 0 代替判断 $x > a$ 。如果 $x = a$ ，则 $x - a - 1 = -1$ ，符号位为 1。只有 $x > a$ 时，符号位为 0。

```
int x_minus_a=x+~a;
int x_minus_b=x+~b;
```

若 x, a 符号相同，则必有 $x - a - 1$ 不为负（规避 $x = a$ 的情况）。 x 和 a 同号，并且 $x > a$ 时， xa_same_sign 为 1。

```
int xa_same_sign=! (sign_x^sign_a)&!(x_minus_a >> 31);
int xb_same_sign=! (sign_x^sign_b)&!(x_minus_b >> 31);
```

合并，得到异号和同号两种分支的严格大于。

```
int x_bigger_a=xa_diff_sign | xa_same_sign;
int x_bigger_b=xb_diff_sign | xb_same_sign;
```

判断相等关系

```
int eq_a=!(x^a);
int eq_b = !(x ^ b); //判断端点是否相等
```

定义大于等于和小于等于，最终能判断 $(x \geq a \text{ 且 } x \leq b)$ 或 $(x \geq b \text{ 且 } x \leq a)$

```
int x_ge_a = x_bigger_a | eq_a; // x >= a
int x_ge_b = x_bigger_b | eq_b; // x >= b
int x_le_a = !x_bigger_a; // x <= a (不严格大于等价于小于等于)
int x_le_b = !x_bigger_b; // x <= b
return (x_ge_a & x_le_b) | (x_ge_b & x_le_a);
}
```

13、mul5Sat

使用位移和加法实现乘法，然后利用符号位，分别判断是否左移溢出和加法溢出。

```
* mul5Sat - return x*5, and if x*5 overflow, change the result to
* INT_MAX(0x7fffffff) or INT_MIN(0x80000000) correspondingly
* Examples: mul5Sat(1) = 0x5, mul5Sat(0x40000000) = 0x7fffffff
int mul5Sat(int x) {
    int x4=x<<2; //位移构造 x*4
    int x5=x4+x;
```

判断是否溢出。

```
int shiftflow=!((x4>>2)^x); //看是否能逆向还原，能则说明无溢出
int addflow=!(((~(x^x4))&(x^x5))>>31); //加法溢出：两加数同号+结果与加
数异号时，为 1
int isflow=~(shiftflow|addflow)+1; //两个情况的溢出分支的合并
```

构造掩码，使得溢出时根据输入符号输出 $tmin$ 或 $tmax$ 。

```
int mask=x>>31; //将 0/1 变成全 0/1 的掩码。x 负，111111，x 正，000000
int tmin=1<<31; //100000...
int tmax=~tmin;
int flownumber=mask^tmax; //x 非负，mask=0, 0^tmax=tmax; x 负，0xffffffff^
tmax= ~tmax=tmin
```

没溢出，输出 $x5$ ；溢出了，根据对应情况判断输出 $tmax$ 还是 $tmin$

```
return ((~isflow)&x5)|(isflow&flownumber);
```

```
}
```

14、classifyAdd3

先把有符号整数转换为无符号整数，再利用每次 32 位加法产生的进位，判断精确和跨越了多少次 2^{32} ，从而确定其上溢、下溢或正常。

```
* classifyAdd3 - classify the exact mathematical sum x+y+z.  
* Return 1 if the sum is greater than INT_MAX, -1 if it is less than  
* INT_MIN, and 0 otherwise. You may not use a wider integer type.
```

符号位翻转与偏置转换， $tmin=0x80000000$ ，与其异或会翻转最高位，其他位保持不变。从无符号数值的角度看，相当于加了 2^{31}

即把有符号数的区间 $[-2^{31}, 2^{31}-1]$ 映射到了 $[0, 2^{32}-1]$

```
int classifyAdd3(int x, int y, int z) {  
    int tmin=1<<31;  
    int ux=x^tmin;  
    int uy=y^tmin;  
    int uz=z^tmin;
```

此时偏置后的总和为 $ux+uy+uz = x+y+z+3*2^{31}$ ，原来的溢出判断转化为进位次数问题。最高位进位 $c1$ 的判断依据是：两个 1 相加必然进位；只有一个 1 时，若相加后最高位变为 0，说明低位来的进位造成了输出进位。

```
int add=ux+uy;//// 第一次无符号加法  
int c1 = (((ux&uy)|((ux ^uy) & ~add)) >> 31) & 1;  
int sum=add+uz;  
int c2 = (((add & uz) | ((add^ uz) & ~sum)) >> 31) & 1;
```

根据进位总数判断结果：

当 $x+y+z < -2^{31}$, $sum < 2^{32}$ ，不会有进位， $c1+c2=0$ ，返回 -1

当 $x+y+z$ 位于正常范围内, 1 个进位, $c1+c2=1$ ，返回 0

当 $x+y+z$ 大于 INT_MAX , 2 个进位, $c1+c2=2$, 返回 1

```
int under = !(c1 + c2); //under=1 表示无任何进位，真实和小于 tmin  
int over = c1 & c2; //over=1 表示两次进位，大于 INT_MAX  
return over + (~under + 1);  
}
```

15、floatScaleThreeHalves

本题不能将 $f/2$ 的位模式与 f 的位模式直接整数相加，因为整数加法不符合浮点数的符号位、指数位和尾数位编码规则。

正确做法是先计算完整有效数的 3 倍，再根据规格化范围除以 2 或 4，并在除法后执行最近偶数舍入。

单精度浮点数包含 1 位符号位、8 位指数位和 23 位尾数位；规格化数的完整有效数为 $frac|0x800000$ ，非规格化数的有效数为 $frac$ 。

当 $exp==0xff$ 时，输入为 NaN 或无穷大，应原样返回。

当 $exp==0$ 时，输入为非规格化数或正负零。直接计算 $triple=3*frac$ ，再右移一位并按最近偶数规则舍入。

若结果第 23 位变为 1，该位会落入指数字段最低位，结果自动成为偏置指数为 1 的规

格化数。

对普通规格化数，代码先补上隐含的最高位 1，再计算 $\text{triple} = 3 * \text{sig}$ 。

若 $\text{triple} < 2^{25}$ ，右移 1 位，指数不变；若 $\text{triple} \geq 2^{25}$ ，右移 2 位，指数加 1。

随后根据余数执行最近偶数舍入；若舍入后有效数达到 2^{24} ，需要再次右移并增加指数。

若指数达到 255，说明结果超出有限表示范围，应返回同号无穷大。

```
* floatScaleThreeHalves - Return bit-level equivalent of expression
f*3/2
unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned
    sign_mask=0x80000000u,exp_mask=0x7f800000u,frac_mask=0x7fffffu;
    unsigned sign=uf&sign_mask,frac=uf&frac_mask;
    unsigned exp=(uf&exp_mask)>>23;
    unsigned triple;
    unsigned q;
    if (exp == 0xffu) { //exp 全是 1，值为 255，为 NAN 或无穷大
        return uf;
    }
    if (exp == 0u) { //指数位为 0，非规约数或正负 0
        triple = frac + frac + frac;
        q = triple >> 1;
        q=q+((triple & 1u) & (q & 1u)); // triple 为奇数表示有 0.5; q 为奇数表示
        //低位整数是奇数，此时才加 1。
        return sign|q; //如果 q 的第 23 位变成 1，该位正好是指数字段最低位，所以结果
        //会自动成为指数为 1 的规格化数。
    }
    unsigned sig = frac|0x8000000u;
    triple = sig + sig + sig; //规格化数：补上隐含的 1
    unsigned shift;
    unsigned remainder;
    unsigned half;
    if (triple >= 0x2000000u) {
        shift = 2;
        exp = exp + 1; //目标有效数本来是 triple/2。如果 triple/2 >= 2^24，即
        //triple >= 2^25，就要改成 triple/4，并让指数加 1。
    }
    else {
        shift = 1;
    }
    q = triple >> shift;
    remainder = triple & ((1u << shift) - 1u); //得到余数
    half = 1u << (shift - 1u);
    if ((remainder > half) || ((remainder == half) && (q & 1u))) {
        q = q + 1; //最近偶数舍入
    }
}
```

```

}
if (q & 0x1000000u) {
    q = q >> 1;
    exp = exp + 1; // 舍入后 q 可能达到 2^24, 需要再右移一位, 指数加 1。
}
/* 指数达到 255, 结果为同号无穷大 */
if (exp >= 0xffu) {
    return sign | 0x7f800000u;
}
return sign | (exp << 23) | (q & 0x7fffffu);
}

```

16、floatRoundEven

将一个单精度浮点数舍入为最近的整数。若该浮点数恰好位于两个整数的正中点，则按照最近偶数舍入规则，选择偶数整数。

单精度浮点数共 32 位，0-22 是尾数位，23-30 是指数位，31 是符号位。

首先提取这三个部分

```

unsigned floatRoundEven(unsigned uf) {
    unsigned sign=(uf>>31)&1u;
    unsigned biased_exp=(uf>>23)&0xFFu;
    unsigned frac=uf&0x7fffff;

```

当指数位全为 1，即 biased_exp == 0xff 时，该浮点数为 NaN 或无穷大，直接返回

```

    if(biased_exp==0xff)
        return uf;

```

单精度浮点数的指数偏置值为 127，因此真实指数为

```

int exp=(int)biased_exp-127;

```

根据指数判断是否需要舍入。当 $E \leq -2$ 时，绝对值小于 0.5，舍入为 0 并保留符号；当 $E \geq 23$ 时，所有尾数位都位于整数部分，浮点数本身已经是整数，因此直接返回原位模式。

```

if(exp<-1)
    return sign<<31;
if(exp>=23)
    return uf;

```

从 24 位有效数中取出整数部分 q:

根据被丢弃部分判断是否进位;

如果 q == 0，返回带符号的零;

用 $q \ll \text{shift}$ 重新构造有效数;

若超过 24 位，再右移并增加指数。

```

int shift=23-exp; // 当指数为 exp 时，低 23-exp 位位于小数点之后，是小数部分。
unsigned val=(1<<23)|frac; // 24 位有效数
// 最近偶数舍入
unsigned round=val&((1u<<shift)-1u); // 要被丢弃的部分
unsigned half =1u<<(shift-1);
unsigned q=val>>shift; // 整数部分

```

```

if (round>half||((round==half&&(q & 1u))) {
    q = q + 1u;
}
if(q==0u){
    return sign<<31;//整数结果为 0，必须返回带符号零
}
val = q << shift;//根据舍入后的整数重新构造，保证原来的小数位全部清零
if(val&(1u<<24)){//进位溢出，第 24 位变成了 1
    val=val>>1;
    biased_exp+=1u;//如果舍入后的有效数达到 2^24，说明 24 位字段放不下，需要
    //有效数右移一位，指数加 1。
}
frac = val & 0x7fffffu;
return (sign<<31)|(biased_exp<<23)|frac;
}

```

17、float_i2f

一个 int 有 32 位，其中有 31 位可以用以表示精度，而 float 的尾数位有 23 位，所以要将 int 的精度位右移 8 位，得到 float 的尾数位。被丢掉的低位和保留的最低位要根据“四舍六入五成双”的原则判断是否舍入。

根据 float 的 sign、exp、frac 三个部分，思路是：取符号→特殊处理 0 和 INT_MIN→用无符号数保存绝对值→找最高位的 1（定指数）→左对齐→右移 8 位取尾数并根据低 8 位舍入→处理进位→合并。

```

* float_i2f - Return bit-level equivalent of expression (float) x.
* Result is returned as unsigned int, but
* it is to be interpreted as the bit-level representation of a
* single-precision floating point values.
unsigned float_i2f(int x) { // 整数转为浮点数
    unsigned sign = (unsigned)x & 0x80000000u;
    unsigned ax, exp, frac, rem;
    int x_exp;
    if (x == 0)
        return 0;
    if (x == (int)0x80000000)
        return 0xc0000000u; // INT_MIN = -2^31, 偏置指数 158, 带符号位为
    // 0xc0000000
    ax = x < 0 ? (unsigned)(-x) : (unsigned)x;
    x_exp = 31;
    while ((ax >> x_exp) == 0)
        x_exp--;
    exp = (unsigned)x_exp + 127u;
    ax <<= (31 - x_exp); // 最高位 1 对齐到 bit 31
    frac = (ax >> 8) & 0x7fffffu; // 尾数部分
}

```

```

rem = ax & 0xffu; //被舍入部分
frac += (rem > 0x80u) ||
        ((rem == 0x80u) && (frac & 1u)); //最近偶数舍入
if (frac & 0x800000u) {
    frac &= 0x7fffffu;
    exp += 1u; //舍入进位, 指数加 1
}
return sign | (exp << 23) | frac;
}

```

18、bitCount

统计类，由于不能用循环，需要用位运算实现并行计算，想到先分块，使得块内数值等于原块中 1 的个数，再合并。构造周期重复的掩码，提取特定部分的数值。

为了使常量在 0x00~0xFF 之间，而掩码像 0x55555555 需要利用周期重复性，用移位自我复制，最终低位保留结果。

```

* bitCount - return count of number of 1's in the binary representation
of x
* Examples: bitCount(5) = 2, bitCount(7) = 3
* Legal ops: ! ~ & ^ | + << >>
int bitCount(int x) { //把 32 位切成若干小块，每块各自数 1 的个数，再两两归并
    int m1, m2, m4, m8, m16;
    m1 = 0x55; //0000 0000 0101 0101
    m1 = m1 | (m1 << 8); //0x00005555
    m1 = m1 | (m1 << 16); //0x55555555, 交替 0/1; 每 2 位一组独立相加，组内 1 的个数保存在这 2 位中，结果范围 0~2，不会溢出到邻组。
    m2 = 0x33; // 0011 0011, 2 位一块 → 4 位一块
    m2 = m2 | (m2 << 8);
    m2 = m2 | (m2 << 16); //将相邻两个 2 位计数合并，每 4 位保存一个 0~4 的计数。
    m4 = 0x0F;
    m4 = m4 | (m4 << 8);
    m4 = m4 | (m4 << 16); //将 4 位计数合并到每个字节的低 4 位。
    m8 = 0xFF;
    m8 = m8 | (m8 << 16); //0x00FF00FF
    m16 = 0xFF;
    m16 = m16 | (m16 << 8); //0x0000FFFF
    x = (x & m1) + ((x >> 1) & m1); //交错开来，用 mask 使得每一块和自己的另一半相加
    x = (x & m2) + ((x >> 2) & m2); //把 2 位作为 1 块
    x = (x & m4) + ((x >> 4) & m4);
    x = (x & m8) + ((x >> 8) & m8);
    x = (x & m16) + ((x >> 16) & m16);
    return x;
}

```

19、bitReverse

第 1 层：相邻 1 位互换(每 2 位内镜像)，mask= 0x55555555，0101…取每块的 1 位

第 2 层：相邻 2 位互换(每 4 位内镜像)，mask=0x33333333，0011…取每块的 2 位

第 3 层：相邻 4 位互换(每字节内镜像)，mask=0x0F0F0F0F，取每字节的低 4 位

第 4 层：相邻 8 位互换(字节序反转)，mask=0x00FF00FF，取第 0、2 字节

第 5 层：相邻 16 位互换(高低半字对调)，取低 16 位

$x = ((x \gg w) \& m) | ((x \& m) \ll w);$ // $x \gg w$ 把块的左半搬到右边， $x \& m$ 取出块的右半再搬到左边，两者 OR 合并，即块内左右互换。掩码保证逻辑位移，在移动时不会有像负数算术右移补的 1 等干扰。

上述掩码均通过小常量移位并按位或复制得到：先在字节内形成重复模式，再复制到高位半区。

```
* bitReverse - Reverse bits in an 32-bit integer
*   Examples: bitReverse(0x80000004) = 0x20000001
*               bitReverse(0x7FFFFFFF) = 0xFFFFFFFF
*   Legal ops: ! ~ & ^ | + << >>
int bitReverse(int x){
    int m1, m2, m4, m8, m16, t;
    m16=0xff|(0xff<<8);//0x0000ffff
    m8=0xff|(0xff<<16);//0x00ff00ff
    t=0x0f|(0x0f<<8); m4=t|(t<<16);//0x0f0f0f0f
    t=0x33|(0x33<<8); m2=t|(t<<16);//0x33333333
    t=0x55|(0x55<<8); m1=t|(t<<16);//0x55555555
    x = ((x >> 1) & m1) | ((x & m1) << 1); //相邻一位互换
    x = ((x >> 2) & m2) | ((x & m2) << 2); //相邻两位互换
    x = ((x >> 4) & m4) | ((x & m4) << 4); //相邻四位互换
    x = ((x >> 8) & m8) | ((x & m8) << 8); //相邻八位互换
    x = (x << 16) | (x >> 16); //相邻 16 位互换
    return x;
}
```

实验总结和收获：

通过本次 DataLab 实验，我进一步理解了补码、位运算和浮点数的底层表示。左移和右移能实现乘除，还能配合掩码完成字段提取、位段交换、并行计数和规格化调整；按位与、按位或、异或和逻辑非可以把条件判断转化为无分支运算，从而在有限操作次数内完成复杂逻辑。

实验过程中，我最主要的问题是对溢出和特殊值考虑不足。直接进行整数加减法可能超出有符号整数范围，负数右移也会产生与逻辑右移不同的高位补位；浮点数中的正负零、非规格化数、无穷大、NaN 以及中点舍入也都需要单独处理。后续解题时，应先明确数值范围和边界条件，再设计通用公式，并用极端值验证。

在方法层面，我掌握“拆分字段—构造掩码—分治压缩—处理进位或舍入—重新组装”的解题路径。尤其是二分折半、并行计数和有效数移位等方法。今后还需加强对 C 语言移位规则、补码运算和浮点规格化过程的理解，从计算机底层数据表示的角度分析和验证问题。

